IT 220 — Database Management System · Lab Manual (Student Workbook)

BIM 4th Semester · 16 weekly practicals · **for complete beginners**

> **Read this first.** This manual assumes you have **never** installed a database, never written SQL, and have never seen a command line. Every step tells you exactly what to click, type, and expect to see. If a step doesn't work, read the "**If it goes wrong**" box under it before asking for help. Do the labs **in order** — each one builds on the last.
>
> **One running example.** Across all 16 labs we build **one** database — a small college called **Model College** (students, teachers, courses, enrolments, fee payments). By Lab 16 you will have designed it, built it, queried it, secured it, and backed it up. This is also your **project**.

What this lab is (and is not)

This lab is hands-on SQL. From Lab 4 to Lab 15 you sit at a computer, type SQL, press Go, and look at the result. Every lab produces something you can screenshot.

What is *not* in this lab: relational algebra notation (σ, π, ⋈), relational calculus, and pen-and-paper normalization. Those are **Unit 3 and Unit 4 theory** and are taught and practised in your **lecture classes**, not here — they are exam topics, not lab work. The one design lab (Lab 3) exists only because your project needs a schema before you can build it.

The tools you will use (install once, in Lab 1)

Tool	What it is	Why we use it
XAMPP	A free bundle that installs the **MySQL/MariaDB** database + a web server on your own computer	One installer gives you everything; works on Windows/Mac/Linux
MySQL / MariaDB	The actual **database server** (the "DBMS")	This is the software the whole course is about
phpMyAdmin	A **web page** (opens in your browser) that lets you click through databases and run SQL	Beginner-friendly — you see tables and results visually
dbdiagram.io	A free website for drawing **ER diagrams**	Used once, in Lab 3, to design the schema

> **What is "SQL"?** SQL (say "sequel" or "S-Q-L") is the language you type to talk to a database — to create tables, put data in, and ask questions of the data. You start writing it in **Lab 1** and never stop.

How to use this manual with your lab report

For **every** lab you must keep evidence for your final **lab report** (see *IT220_Lab_Report_Format.md*). After each lab, save: 1. The **SQL** you ran (copy the text). 2. A **screenshot** of the result on *your* screen. 3. A **one-line note** in your own words of what it did.

Take screenshots as you go — do **not** try to recreate them at the end of the semester.

PART 1 — Getting set up (Labs 1–3)

Lab 1 (Week 1) — Install everything & run your first query

Goal: Get XAMPP installed, start the database server, open phpMyAdmin, and run your first query to see live data. You are **not** expected to understand the SQL yet — just see it work.

Before you start

- A laptop where you can install software (you may need the admin/login password).
- ~30 minutes and an internet connection for the download.

Steps

1. Download XAMPP. - Open your browser, go to `https://www.apachefriends.org`. - Click the download for **your operating system** (Windows / macOS / Linux). The file is large (~150 MB); wait for it to finish.

2. Install XAMPP. - Open the downloaded file and click **Next** through the installer, accepting the defaults. - If Windows shows a security/firewall popup, click **Allow**. - Finish and let it open the **XAMPP Control Panel** (Windows) or **manager-osx** (Mac).

3. Start the database. - In the XAMPP Control Panel, find the row that says **MySQL** (on some versions it says **MariaDB** — same thing for us) and click its **Start** button. - Also click **Start** next to **Apache** (this runs the web server that phpMyAdmin needs). - Both rows should turn **green**.

4. Open phpMyAdmin. - Open your browser and go to `http://localhost/phpmyadmin`. - You should see a page with a list of databases on the left and tabs (Databases, SQL, Status…) across the top. **This is your window into the database server.**

5. Make a practice database to look at. - Click the **SQL** tab at the top. - A big empty text box appears. **Copy the block below exactly** and paste it in, then click **Go** (bottom right).

```
CREATE DATABASE demo_shop;
USE demo_shop;
CREATE TABLE product (
  id    INT PRIMARY KEY,
  name  VARCHAR(50),
  price INT
);
INSERT INTO product VALUES (1, 'Wai Wai Noodles', 20);
INSERT INTO product VALUES (2, 'Dairy Milk', 100);
INSERT INTO product VALUES (3, 'Frooti', 35);
```

- You should see a green "**success**" message. On the left, a new database **demo_shop** appears.

6. Run your first queries. - Still on the **SQL** tab, clear the box, type each of these, and click **Go**:

```
SELECT * FROM product;
```

```
SELECT name, price FROM product;
```

```
SELECT name FROM product WHERE price > 30;
```

- The first shows all 3 products, the second only two columns, the third only the items over Rs 30. **Congratulations — you just queried a database.**

What you should see

A results grid with 3 rows (Wai Wai, Dairy Milk, Frooti), then a 2-column version, then 2 rows. Take a **screenshot** of the first and the third.

If it goes wrong

- **MySQL won't turn green / stops immediately (Windows):** usually another program uses its port. In the Control Panel click **Config → my.ini**, or simplest: close Skype and any other database, then Start again. If it still fails, restart the computer and try once more.
- **`http://localhost/phpmyadmin` shows "not found":** Apache isn't started — go back and Start Apache too.
- **"Access denied":** on a fresh XAMPP the username is **`root`** with an **empty password** — don't type a password.

What you learned

A **database server** (MySQL) runs in the background; **phpMyAdmin** is just a friendly window onto it; and **SQL** is the text you send to make it do things. `SELECT * FROM product` means "show me every row from the product table"; adding `WHERE` filters which rows come back.

Lab 2 (Week 2) — Inside the DBMS: databases, catalog, users

Goal: Understand the *structure* of a database server — the difference between a **database**, a **table**, and a **row** — query the server's own catalog, and create a **user** with limited permissions. This makes Unit 1's "three-schema architecture" and "DBA" concrete.

Steps

1. See the levels. In phpMyAdmin, look at the left panel: - The **server** holds many **databases** (you see `demo_shop` and some system ones). - Click **demo_shop** → it holds **tables** (you see `product`). - Click **product** → it holds **rows** (your 3 items) and **columns** (id, name, price).

Write this hierarchy in your report: **Server → Database → Table → Row/Column.**

2. Query the catalog (data about data). The server keeps a database describing *itself*. Run these on the **SQL** tab:

```
-- every table in your demo_shop database
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'demo_shop';

-- every column of the product table, with its data type
SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_schema = 'demo_shop' AND table_name = 'product';

-- how many databases exist on this server
SELECT COUNT(*) AS total_databases FROM information_schema.schemata;
```

The first returns one row: `product`. **This is metadata — "data about the data".**

3. Create a database user (the DBA's job). - Click the **User accounts** tab at the top. - Click **Add user account**. - User name: `college_clerk` · Host name: choose **Local** · Password: type `clerk123` (twice). - **Do not** tick "Global privileges." Scroll down and click **Go**.

4. Give that user limited rights. - Back on **User accounts**, click **Edit privileges** next to `college_clerk`. - Click the **Database** tab, choose `demo_shop`, click **Go**. - Tick only **SELECT** and **INSERT** (not DROP or DELETE), then **Go**. - You have just made a user who can *read and add* data but cannot *delete tables*. That is what a **DBA (Database Administrator)** does — hand out only the rights each person needs.

5. Confirm it with SQL:

```
SHOW GRANTS FOR 'college_clerk'@'localhost';
```

What you should see

The `information_schema` queries returning `product` and the column list, and `SHOW GRANTS` listing only SELECT and INSERT on `demo_shop`. Screenshot the catalog query and the grants.

If it goes wrong

- **`information_schema` query returns nothing:** check the database name is spelled exactly `demo_shop` and is inside quotes.
- "**Access denied**" **creating the user:** you are not logged in as `root` — log out and back in.

What you learned

The DBMS separates **physical storage**, the **logical tables** you see, and the **views/permissions** each user gets — that layering is the **three-schema architecture**. The **catalog** (`information_schema`) is the DBMS's own record of every database, table, and column. A **DBA** manages users and their privileges with `GRANT` / `REVOKE`.

Lab 3 (Week 3) — Design the Model College database

Goal: Before building any table, *design* the database you will use for the rest of the semester and for your project. This is the **only** lab without SQL — you need a schema before you can create it in Lab 4.

> **Vocabulary (don't skip):** - **Entity** = a *thing* we store data about (a Student, a Course). Becomes a table. - **Attribute** = a *fact* about an entity (a student's name, roll no). Becomes a column. - **Primary key (PK)** = the attribute that *uniquely* identifies one row (roll_no for a student). - **Foreign key (FK)** = a column pointing at another table's PK — it links the tables. - **Relationship** = how entities connect ("a student *enrols in* a course"). - **Cardinality** = how many: one-to-one (1:1), one-to-many (1:N), many-to-many (M:N).

Steps

1. List the entities of Model College and their attributes: - **Student** (roll_no [PK], name, address, phone, semester) - **Teacher** (teacher_id [PK], name, department, phone) - **Course** (course_code [PK], title, credit_hours) - **FeePayment** (receipt_no [PK], amount, paid_on) — belongs to a Student - **Enrolment** — a student takes a course in a semester (this connects Student and Course)

2. Decide the relationships and cardinality: - A **Student** *enrols in* many **Courses**, and a **Course** has many **Students** → **M:N**. - A **Teacher** *teaches* many **Courses**, a Course is taught by one Teacher → **1:N**. - A **Student** *makes* many **FeePayments**, each payment belongs to one Student → **1:N**.

3. Resolve the M:N. A student takes many courses *and* a course has many students. Neither table can hold this. The rule: **every M:N becomes a new table in the middle** — a **junction table**. Ours is **Enrolment**, and its primary key is the *pair* `(roll_no, course_code)` — a **composite primary key**, because a student cannot enrol in the same course twice.

4. Draw it in a tool. - Go to **`https://dbdiagram.io`** → click **Go to app** (no login needed to start). - On the left code panel, delete the sample and type this (it draws the diagram on the right):

```
Table Teacher {
  teacher_id int [pk]
  name varchar
  department varchar
  phone varchar
}
Table Student {
  roll_no int [pk]
  name varchar
  address varchar
  phone varchar
  semester int
}
Table Course {
  course_code varchar [pk]
  title varchar
  credit_hours int
  teacher_id int [ref: > Teacher.teacher_id]   // 1:N teacher→course
}
Table Enrolment {
  roll_no int [ref: > Student.roll_no]
```

```
  course_code varchar [ref: > Course.course_code]
  semester int
  grade varchar
  indexes {
    (roll_no, course_code) [pk]   // composite primary key
  }
}
Table FeePayment {
  receipt_no int [pk]
  roll_no int [ref: > Student.roll_no]          // 1:N student→payment
  amount int
  paid_on date
}
```

- The diagram appears on the right with lines connecting the tables. **Export it: Export → PNG.**

5. Write out the schema in text — this is your build plan for Lab 4:

```
Student(roll_no PK, name, address, phone, semester)
Teacher(teacher_id PK, name, department, phone)
Course(course_code PK, title, credit_hours, teacher_id FK→Teacher)
Enrolment(roll_no FK→Student, course_code FK→Course, semester, grade)
          PK = (roll_no, course_code)
FeePayment(receipt_no PK, roll_no FK→Student, amount, paid_on)
```

What you should see

A clean 5-table ER diagram with Enrolment sitting between Student and Course. Export the PNG — it goes straight into your report.

If it goes wrong

- **Red error in dbdiagram:** you mistyped a keyword — check every `[pk]`, `ref:` and the curly braces `{ }`. Copy the block again carefully.

What you learned

You **model the database before typing SQL**. Entities become tables, attributes become columns, relationships become foreign keys, and every **M:N** relationship needs a **junction table** with a **composite primary key**. Next week you build exactly this in SQL.

PART 2 — Building and querying in SQL (Labs 4–12)

From here on, every lab is SQL you type and run.

> **Where you type SQL from now on:** phpMyAdmin → click your database → the **SQL** tab → type → **Go**. You can run one statement or several at once.

Lab 4 (Week 4) — Create the database and tables (DDL)

Goal: Build the Lab 3 schema for real using **DDL** (Data Definition Language: `CREATE`), with proper keys and constraints.

Steps

1. Create the database. phpMyAdmin → **SQL** tab → run:

```
CREATE DATABASE model_college;
USE model_college;
```

> From now on, click **model_college** in the left panel *before* using the SQL tab, so your statements go to the right database.

2. Create the tables (run this whole block). Read the comments — they explain each **constraint**:

```
CREATE TABLE Teacher (
  teacher_id  INT PRIMARY KEY,        -- unique id for each teacher
  name        VARCHAR(60) NOT NULL,   -- NOT NULL = must have a value
  department  VARCHAR(40),
  phone       VARCHAR(15)
);

CREATE TABLE Student (
  roll_no   INT PRIMARY KEY,
  name      VARCHAR(60) NOT NULL,
  address   VARCHAR(60),
  phone     VARCHAR(15),
  semester  INT CHECK (semester BETWEEN 1 AND 8)  -- CHECK = only 1..8
);

CREATE TABLE Course (
  course_code  VARCHAR(10) PRIMARY KEY,
  title        VARCHAR(60) NOT NULL,
  credit_hours INT DEFAULT 3,         -- DEFAULT if none given
  teacher_id   INT,
  -- FOREIGN KEY = must point to a real teacher
  FOREIGN KEY (teacher_id) REFERENCES Teacher(teacher_id)
);

CREATE TABLE Enrolment (
  roll_no      INT,
  course_code  VARCHAR(10),
  semester     INT,
  grade        VARCHAR(2),
  PRIMARY KEY (roll_no, course_code),           -- composite key
  FOREIGN KEY (roll_no) REFERENCES Student(roll_no),
  FOREIGN KEY (course_code) REFERENCES Course(course_code)
);

CREATE TABLE FeePayment (
  receipt_no INT PRIMARY KEY,
  roll_no    INT,
```

```
  amount     INT NOT NULL,
  paid_on    DATE,
  FOREIGN KEY (roll_no) REFERENCES Student(roll_no)
);
```

3. Inspect what you built:

```
SHOW TABLES;              -- lists all 5 tables
DESCRIBE Student;         -- shows Student's columns, types, keys
DESCRIBE Enrolment;       -- note the composite PRIMARY KEY
```

4. Try to break a constraint on purpose (this is the point of the lab — see the database defend itself). Each of these should give an **error**:

```
-- CHECK fails: semester 99
INSERT INTO Student VALUES (1,'Test','KTM','98',99);

-- FK fails: no teacher 999
INSERT INTO Course VALUES ('X1','Fake',3,999);

-- NOT NULL fails: name is null
INSERT INTO Student VALUES (2,NULL,'KTM','98',2);
```

Screenshot one of these errors — it proves the constraint works.

What you should see

Five tables listed on the left under `model_college`; `DESCRIBE` output showing PRI on the key columns; and a red error message from each deliberate mistake in step 4.

If it goes wrong

- "**Table already exists**": you ran the block twice. Either skip it, or run `DROP TABLE Enrolment, FeePayment, Course, Student, Teacher;` and start again (children first).
- `CHECK` **seems ignored:** very old MariaDB/MySQL versions parse but don't enforce CHECK. Note it in your report and move on — the other constraints still work.

What you learned

DDL builds structure. **Constraints** (`PRIMARY KEY`, `FOREIGN KEY`, `NOT NULL`, `CHECK`, `DEFAULT`) are the database *protecting itself* from bad data — this is why you can never enrol a student who doesn't exist.

Lab 5 (Week 5) — Put data in (INSERT)

Goal: Populate all five tables with `INSERT`, in the correct order, and read the data back.

Steps

1. Insert the parent tables first (Teacher, Student, Course — nothing points at them yet):

```
INSERT INTO Teacher VALUES
 (1,'Ram Thapa','Computer','9841000001'),
 (2,'Gita Sharma','English','9841000002'),
 (3,'Sunil Basnet','Management','9841000003');

INSERT INTO Student VALUES
 (101,'Sita Rai','Bhaktapur','9801000101',2),
 (102,'Hari Karki','Lalitpur','9801000102',2),
 (103,'Mina Gurung','Kathmandu','9801000103',4),
 (104,'Bikash Lama','Pokhara','9801000104',4),
 (105,'Anita Shrestha','Kathmandu','9801000105',2);

INSERT INTO Course VALUES
 ('CMP101','Programming',3,1),
 ('ENG101','English',3,2),
 ('DBM220','Database Management',3,1),
 ('MGT210','Management',3,3);
```

2. Now the child tables (they reference the rows you just made):

```
INSERT INTO Enrolment VALUES
 (101,'CMP101',2,'A'),
 (101,'ENG101',2,'B'),
 (102,'CMP101',2,'A'),
 (102,'ENG101',2,'C'),
 (103,'DBM220',4,'B'),
 (104,'DBM220',4,NULL),
 (104,'MGT210',4,'A'),
 (105,'CMP101',2,NULL);

INSERT INTO FeePayment VALUES
 (5001,101,15000,'2026-01-10'),
 (5002,102,15000,'2026-01-12'),
 (5003,103,12000,'2026-01-15'),
 (5004,101,5000,'2026-02-10');
```

3. Insert only some columns — the rest take their DEFAULT or NULL:

```
INSERT INTO Course (course_code, title, teacher_id)
VALUES ('STA201','Statistics',3);

-- credit_hours filled itself with 3
SELECT * FROM Course WHERE course_code = 'STA201';
```

4. Read everything back:

```
SELECT * FROM Student;
SELECT * FROM Enrolment;
SELECT * FROM FeePayment;
```

5. Prove the order mattered. Try inserting a child whose parent doesn't exist:

```
-- ERROR: student 999 doesn't exist
INSERT INTO Enrolment VALUES (999,'CMP101',2,'A');
```

What you should see

5 students, 8 enrolments, 4 payments; `STA201` with credit_hours = 3 even though you never typed it; and a foreign-key error on step 5. Screenshot the `SELECT * FROM Student;` result and the error.

If it goes wrong

- "**Cannot add or update a child row: a foreign key constraint fails**": you inserted a row that points to something that doesn't exist yet. Insert **parents first** (Teacher, Student, Course) *then* children (Enrolment, FeePayment) — the order above already does this.
- "**Duplicate entry for key PRIMARY**": you ran the INSERT twice. Skip it, or delete the rows first.

What you learned

`INSERT INTO … VALUES` adds rows. You may insert **all** columns positionally, or **named** columns only — the rest fall back to `DEFAULT` or `NULL`. Foreign keys force an insert **order**: parents before children.

Lab 6 (Week 6) — Asking questions: SELECT, WHERE, ORDER BY, DISTINCT

Goal: The single most-used SQL statement. Filter rows, choose columns, sort, and remove duplicates.

Steps — run these one at a time and read each result

```
-- 1. All columns, all rows
SELECT * FROM Student;

-- 2. Only the columns you want
SELECT name, semester FROM Student;

-- 3. Rename a column in the output with AS
SELECT name AS student_name, semester AS sem FROM Student;

-- 4. WHERE = filter rows
SELECT name FROM Student WHERE semester = 2;
SELECT * FROM FeePayment WHERE amount > 12000;

-- 5. Combine conditions with AND / OR / NOT
SELECT name FROM Student WHERE semester = 2 AND address = 'Kathmandu';
SELECT name FROM Student WHERE semester = 2 OR semester = 4;

-- 6. IN = "is any one of these"; BETWEEN = a range
SELECT name FROM Student WHERE address IN ('Kathmandu','Pokhara');
SELECT * FROM FeePayment WHERE amount BETWEEN 10000 AND 15000;

-- 7. LIKE = pattern match. % means "any characters"
SELECT name FROM Student WHERE name LIKE 'S%';      -- starts with S
SELECT name FROM Student WHERE name LIKE '%Rai';    -- ends with Rai
SELECT title FROM Course WHERE title LIKE '%Manage%';

-- 8. ORDER BY = sort. ASC = A→Z (default), DESC = Z→A / high→low
SELECT name, semester FROM Student ORDER BY name ASC;
SELECT * FROM FeePayment ORDER BY amount DESC;
SELECT name, semester FROM Student ORDER BY semester DESC, name ASC;

-- 9. DISTINCT = remove duplicate values
SELECT DISTINCT semester FROM Student;
SELECT DISTINCT address FROM Student;

-- 10. LIMIT = only the first N rows
SELECT * FROM FeePayment ORDER BY amount DESC LIMIT 2;   -- top 2 payments
```

What you should see

Query 4 returns 3 second-semester students (Sita, Hari, Anita); query 5's first form returns only Anita Shrestha; query 7's `LIKE 'S%'` returns only Sita Rai; query 9 returns just `2` and `4`. Screenshot queries 4, 7, and 8.

If it goes wrong

- "**Unknown column 'name' in field list**": you are querying the wrong database or misspelled the column — click `model_college` on the left first, and check with `DESCRIBE Student;`.

- `LIKE 'S%'` **returns nothing:** the `%` must be inside the quotes.

What you learned

`SELECT columns FROM table WHERE condition ORDER BY column` is the shape of nearly every query you will ever write. `WHERE` picks **rows**, the column list picks **columns**, `ORDER BY` sorts, `DISTINCT` de-duplicates, and `LIKE` does pattern matching.

Lab 7 (Week 7) — Summarising: aggregates, GROUP BY, HAVING, NULL

Goal: Collapse many rows into useful numbers — counts, totals, averages — and filter those summaries.

Steps

1. Aggregate functions — many rows in, one number out:

```
SELECT COUNT(*)   AS total_students   FROM Student;
SELECT COUNT(*)   AS total_payments   FROM FeePayment;
SELECT SUM(amount) AS total_collected FROM FeePayment;
SELECT AVG(amount) AS average_payment FROM FeePayment;
SELECT MAX(amount) AS biggest, MIN(amount) AS smallest FROM FeePayment;
```

2. GROUP BY — one summary row *per group*:

```
-- how many students in each semester
SELECT semester, COUNT(*) AS how_many
FROM Student
GROUP BY semester;

-- how many students enrolled in each course
SELECT course_code, COUNT(*) AS students_in_course
FROM Enrolment
GROUP BY course_code;

-- total fees paid by each student
SELECT roll_no, SUM(amount) AS total_paid
FROM FeePayment
GROUP BY roll_no;
```

3. HAVING — filter the *groups*, after grouping:

```
-- only courses with 2 or more students
SELECT course_code, COUNT(*) AS students_in_course
FROM Enrolment
GROUP BY course_code
HAVING COUNT(*) >= 2;

-- only students who have paid more than Rs 15000 in total
SELECT roll_no, SUM(amount) AS total_paid
FROM FeePayment
GROUP BY roll_no
HAVING SUM(amount) > 15000;
```

4. WHERE vs HAVING — run both and compare. `WHERE` runs **before** grouping, `HAVING` **after**:

```
-- WHERE first: only sem-2 enrolments, then group
SELECT course_code, COUNT(*) FROM Enrolment
WHERE semester = 2
GROUP BY course_code;
```

5. NULL handling — NULL means *unknown*, and is never equal to anything:

```
-- correct
SELECT roll_no, course_code FROM Enrolment WHERE grade IS NULL;

-- WRONG: returns nothing
SELECT roll_no, course_code FROM Enrolment WHERE grade = NULL;

SELECT roll_no, course_code FROM Enrolment WHERE grade IS NOT NULL;

SELECT COUNT(*) AS rows_total, COUNT(grade) AS rows_with_grade
FROM Enrolment;
```

Notice the last query: `COUNT(*)` counts rows, `COUNT(grade)` **skips NULLs**.

What you should see

Semester 2 → 3 students, semester 4 → 2. `CMP101` has 3 students. Roll 101 paid 20000 total. The `= NULL` query returns **zero rows** while `IS NULL` returns 2. Screenshot the GROUP BY, the HAVING, and the `= NULL` vs `IS NULL` pair.

If it goes wrong

- "**Expression not in GROUP BY clause**": every column in your SELECT must either be inside an aggregate, or listed in the `GROUP BY`.
- **`HAVING` gives an error:** `HAVING` only works with `GROUP BY` — use `WHERE` for plain row filtering.

What you learned

Aggregates (COUNT/SUM/AVG/MIN/MAX) collapse many rows into one value; **GROUP BY** does it per category; **HAVING** filters those summaries. `WHERE` filters **rows before** grouping, `HAVING` filters **groups after**. Use `IS NULL`, never `= NULL`.

Lab 8 (Week 8) — Combining tables: JOINs

Goal: Your data is split across five tables. A **JOIN** stitches them back together using the foreign keys you designed in Lab 3.

Steps

1. INNER JOIN — only rows that match in both tables:

```
-- each enrolment with the student's real name
SELECT s.name, e.course_code, e.grade
FROM Enrolment e
JOIN Student s ON e.roll_no = s.roll_no;

-- three tables at once: student + course title + grade
SELECT s.name, c.title, e.grade
FROM Enrolment e
JOIN Student s ON e.roll_no = s.roll_no
JOIN Course  c ON e.course_code = c.course_code;

-- each course with its teacher's name
SELECT c.title, t.name AS teacher
FROM Course c
JOIN Teacher t ON c.teacher_id = t.teacher_id;
```

2. LEFT JOIN — keep ALL rows from the left table, even unmatched ones:

```
-- every student, including those with no enrolment (they show NULL)
SELECT s.name, e.course_code
FROM Student s
LEFT JOIN Enrolment e ON s.roll_no = e.roll_no;

-- every student and what they have paid — non-payers show NULL
SELECT s.name, f.amount, f.paid_on
FROM Student s
LEFT JOIN FeePayment f ON s.roll_no = f.roll_no;

-- find the non-payers: LEFT JOIN then keep only the NULLs
SELECT s.roll_no, s.name
FROM Student s
LEFT JOIN FeePayment f ON s.roll_no = f.roll_no
WHERE f.receipt_no IS NULL;
```

3. Self-join — join a table to itself. Find pairs of classmates in the same semester:

```
SELECT a.name AS student_1, b.name AS student_2, a.semester
FROM Student a
JOIN Student b ON a.semester = b.semester AND a.roll_no < b.roll_no;
```

The `a.roll_no < b.roll_no` stops you getting each pair twice (and a student paired with themselves).

4. JOIN + GROUP BY — the combination you will use most:

```
-- how many students in each course, by course TITLE not code
```

```
SELECT c.title, COUNT(*) AS students
FROM Enrolment e
JOIN Course c ON e.course_code = c.course_code
GROUP BY c.title;

-- total fees paid, per student name
SELECT s.name, SUM(f.amount) AS total_paid
FROM Student s
JOIN FeePayment f ON s.roll_no = f.roll_no
GROUP BY s.name;
```

What you should see

The 3-table INNER JOIN lists name + course title + grade together. The LEFT JOIN shows every student, with NULL for any without enrolments. The non-payer query returns roll 104 and 105. Screenshot the INNER JOIN, the LEFT JOIN, and the non-payer query.

If it goes wrong

- "**Column 'name' in field list is ambiguous**": both tables have a `name` column — you must say which one: `s.name`, not `name`.
- **The JOIN returns far too many rows:** you forgot the `ON` condition, so every row matched every row (a *cartesian product*). Always give an `ON`.

What you learned

INNER JOIN keeps only rows that match in both tables; **LEFT JOIN** keeps everything from the left table and fills NULLs where there's no match — which is how you find the "missing" ones. A **self-join** compares rows within one table. Joins work because of the **foreign keys** from Lab 3.

Lab 9 (Week 9) — Subqueries

Goal: Put a query *inside* another query, so one question's answer feeds the next.

Steps

1. Subquery returning one value (use it like a number):

```
-- the single largest payment
SELECT * FROM FeePayment
WHERE amount = (SELECT MAX(amount) FROM FeePayment);

-- students who paid more than the average payment
SELECT roll_no, amount FROM FeePayment
WHERE amount > (SELECT AVG(amount) FROM FeePayment);
```

2. Subquery returning a list — `IN` / `NOT IN`:

```
-- students who have made at least one fee payment
SELECT name FROM Student
WHERE roll_no IN (SELECT roll_no FROM FeePayment);

-- students who have NOT paid anything
SELECT name FROM Student
WHERE roll_no NOT IN (SELECT roll_no FROM FeePayment);

-- courses taught by someone from the Computer department
SELECT title FROM Course
WHERE teacher_id IN (SELECT teacher_id FROM Teacher
                     WHERE department = 'Computer');
```

3. `EXISTS` — true if the inner query finds any row at all:

```
-- students who are enrolled in something
SELECT name FROM Student s
WHERE EXISTS (SELECT 1 FROM Enrolment e WHERE e.roll_no = s.roll_no);

-- students enrolled in nothing
SELECT name FROM Student s
WHERE NOT EXISTS (SELECT 1 FROM Enrolment e WHERE e.roll_no = s.roll_no);
```

4. Subquery in FROM (a derived table) — treat a query's result as a table:

```
SELECT AVG(total_paid) AS average_per_student
FROM (SELECT roll_no, SUM(amount) AS total_paid
      FROM FeePayment
      GROUP BY roll_no) AS totals;
```

What you should see

The `IN` query returns the students who paid (101, 102, 103); `NOT IN` returns 104 and 105 — the same answer you got with a LEFT JOIN in Lab 8. Screenshot the `IN`, the `NOT IN`, and the derived table.

If it goes wrong

- `NOT IN` **returns nothing unexpectedly:** if the inner query returns any NULL, `NOT IN` gives no rows. Add `WHERE roll_no IS NOT NULL` inside it.
- "**Subquery returns more than 1 row**": you used `=` with a subquery that returns a list — use `IN` instead.
- **Derived table error:** a subquery in `FROM` **must** be given an alias (`AS totals`).

What you learned

A **subquery** is a query whose result feeds an outer query — as a single value, a list (`IN`), a test (`EXISTS`), or a whole table (in `FROM`). Many subqueries can also be written as JOINs; both are correct.

Lab 10 (Week 10) — Views

Goal: Save a query under a name and use it like a table.

Steps

1. Create a view:

```
CREATE VIEW paid_students AS
SELECT s.roll_no, s.name, f.amount, f.paid_on
FROM Student s
JOIN FeePayment f ON s.roll_no = f.roll_no;
```

2. Use it exactly like a table:

```
SELECT * FROM paid_students;
SELECT name FROM paid_students WHERE amount > 12000;
SELECT roll_no, SUM(amount) FROM paid_students GROUP BY roll_no;
```

3. Make a second, more useful view — a full enrolment sheet:

```
CREATE VIEW enrolment_sheet AS
SELECT s.roll_no, s.name AS student, c.title AS course,
       t.name AS teacher, e.grade
FROM Enrolment e
JOIN Student s ON e.roll_no = s.roll_no
JOIN Course  c ON e.course_code = c.course_code
JOIN Teacher t ON c.teacher_id = t.teacher_id;

SELECT * FROM enrolment_sheet;
SELECT * FROM enrolment_sheet WHERE grade IS NULL;
```

Notice you just asked a 4-table question in one short line. **That is what views are for.**

4. Views are live, not copies. Change the underlying data and re-query the view:

```
UPDATE Enrolment SET grade = 'B'
WHERE roll_no = 104 AND course_code = 'DBM220';

-- the view already shows B
SELECT * FROM enrolment_sheet WHERE roll_no = 104;
```

5. Updatable vs read-only views. A simple one-table view can be updated through:

```
CREATE VIEW sem2_students AS
SELECT roll_no, name, phone FROM Student WHERE semester = 2;

UPDATE sem2_students SET phone = '9800000000' WHERE roll_no = 105;
SELECT * FROM Student WHERE roll_no = 105;   -- the real table changed
```

But a view with a JOIN or GROUP BY generally **cannot** be updated — try it and note the error:

```
-- expect an error
```

```
UPDATE enrolment_sheet SET grade = 'A' WHERE roll_no = 101;
```

6. Housekeeping:

```
SHOW FULL TABLES WHERE table_type = 'VIEW';   -- list your views
-- DROP VIEW sem2_students;                   -- how you remove one
```

What you should see

`paid_students` and `enrolment_sheet` behaving like tables; the view reflecting the UPDATE immediately; the update through `sem2_students` changing the real `Student` row; and an error when updating the joined view. Screenshot `enrolment_sheet` and the update-through-view result.

If it goes wrong

- "**View's SELECT contains a subquery in the FROM clause**": MySQL does not allow that in a view — simplify the query.
- "**Table 'paid_students' already exists**": you created it twice — use `CREATE OR REPLACE VIEW`.

What you learned

A **view** is a **saved query**, not stored data — it computes its rows fresh every time, so it is always current. Views simplify complex joins, and let you expose only certain columns to a restricted user. Simple views are **updatable**; joined/aggregated ones are read-only.

Lab 11 (Week 11) — Changing data: UPDATE and DELETE

Goal: Modify existing rows safely. This is the lab where carelessness destroys data — so the technique matters as much as the syntax.

> ⚠️ **The most dangerous beginner mistake:** `UPDATE Student SET semester = 1;` with **no WHERE** changes *every single row*. `DELETE FROM Student;` empties the whole table. **Always write the `WHERE` first, then go back and write the `UPDATE` / `DELETE` in front of it.**

Steps

1. The safe habit — SELECT before you UPDATE. Always run the `WHERE` as a `SELECT` first, to see exactly which rows you are about to hit:

```
-- STEP 1: check which rows the WHERE matches
SELECT * FROM Student WHERE roll_no = 105;

-- STEP 2: only now, change them
UPDATE Student SET phone = '9811111111' WHERE roll_no = 105;

-- STEP 3: confirm
SELECT * FROM Student WHERE roll_no = 105;
```

2. More UPDATEs:

```
-- change several columns at once
UPDATE Student SET address = 'Bhaktapur', phone = '9812222222'
WHERE roll_no = 104;

-- update using the existing value (a 10% fee increase)
UPDATE FeePayment SET amount = amount * 1.10 WHERE receipt_no = 5003;

-- fill in a missing grade
UPDATE Enrolment SET grade = 'C'
WHERE roll_no = 105 AND course_code = 'CMP101';

-- update many rows on purpose (note it still has a WHERE)
UPDATE Student SET semester = semester + 1 WHERE semester = 2;
```

Check the last one, then put it back:

```
SELECT roll_no, name, semester FROM Student;
UPDATE Student SET semester = semester - 1 WHERE semester = 3;
```

3. DELETE — same rule:

```
-- add a throwaway row so we can delete it
INSERT INTO Student VALUES (199,'Temp Test','Nowhere','9800000000',1);

-- STEP 1: see what will die
SELECT * FROM Student WHERE roll_no = 199;
-- STEP 2: delete it
```

```
DELETE FROM Student WHERE roll_no = 199;
-- STEP 3: confirm it's gone
SELECT * FROM Student WHERE roll_no = 199;
```

4. Foreign keys protect you from deleting a parent:

```
-- ERROR: rows in Enrolment/FeePayment reference it
DELETE FROM Student WHERE roll_no = 101;
```

To really remove that student you must delete the children first (don't run this on your project data unless you mean it):

```
-- DELETE FROM Enrolment  WHERE roll_no = 101;
-- DELETE FROM FeePayment WHERE roll_no = 101;
-- DELETE FROM Student    WHERE roll_no = 101;
```

5. The safety net. Before any risky change, wrap it in a transaction you can undo (this is a preview of Lab 13):

```
START TRANSACTION;
UPDATE Student SET semester = 1;     -- deliberately missing WHERE!
SELECT roll_no, semester FROM Student;   -- everything is 1 — disaster
ROLLBACK;                                 -- undo it
SELECT roll_no, semester FROM Student;   -- back to normal
```

What you should see

Each UPDATE reporting "N rows affected" — and that N should match what your `SELECT` predicted. The DELETE on roll 101 failing with a foreign-key error. The no-WHERE disaster in step 5 wiping every semester, then `ROLLBACK` restoring it. Screenshot the SELECT-then-UPDATE pair, the FK error, and the rollback recovery.

If it goes wrong

- "**0 rows affected**": your `WHERE` matched nothing — check the value exists with a `SELECT`.
- **You updated everything by accident:** if you were inside a transaction, `ROLLBACK`. If not, restore from your backup (Lab 15). This is why step 5 exists.
- "**Cannot delete or update a parent row**": a child row still references it — delete children first.

What you learned

`UPDATE table SET column = value WHERE condition` and `DELETE FROM table WHERE condition`. The `WHERE` is not optional in practice — without it you hit every row. **SELECT first, then modify.** Foreign keys stop you orphaning child rows, and a transaction gives you an undo button.

Lab 12 (Week 12) — Stored procedures, triggers, indexing · SQL practical test week

Goal: Put logic *inside* the database with a **stored procedure** and a **trigger**, and speed up searching with an **index**.

> **Note:** In phpMyAdmin, when you create a procedure or trigger that contains semicolons, set the **Delimiter** box (below the SQL area) to `$$` first, or use the code exactly as written below.

Steps

1. Stored procedure — a saved block of SQL you call by name:

```
DELIMITER $$
CREATE PROCEDURE students_in_semester(IN sem INT)
BEGIN
  SELECT roll_no, name FROM Student WHERE semester = sem;
END $$
DELIMITER ;
```

Run it:

```
CALL students_in_semester(2);
CALL students_in_semester(4);
```

2. A procedure with an OUT value:

```
DELIMITER $$
CREATE PROCEDURE total_paid_by(IN roll INT, OUT total INT)
BEGIN
  SELECT SUM(amount) INTO total FROM FeePayment WHERE roll_no = roll;
END $$
DELIMITER ;

CALL total_paid_by(101, @t);
SELECT @t AS total_paid_by_101;
```

3. Trigger — SQL that runs *automatically* on an event. Make an audit log of new payments:

```
CREATE TABLE payment_log (
  log_id INT AUTO_INCREMENT PRIMARY KEY,
  receipt_no INT,
  logged_at DATETIME
);

DELIMITER $$
CREATE TRIGGER after_payment
AFTER INSERT ON FeePayment
FOR EACH ROW
BEGIN
  INSERT INTO payment_log (receipt_no, logged_at)
  VALUES (NEW.receipt_no, NOW());
```

```
END $$
DELIMITER ;
```

Test it — insert a payment, then check the log filled itself. **You never touched `payment_log`:**

```
INSERT INTO FeePayment VALUES (5005, 104, 15000, '2026-01-20');
SELECT * FROM payment_log;
```

4. Index — a lookup structure that speeds up searching a column:

```
-- before: see how the database plans the query
EXPLAIN SELECT * FROM Student WHERE name = 'Sita Rai';

CREATE INDEX idx_student_name ON Student(name);

-- after: the plan now uses the index
EXPLAIN SELECT * FROM Student WHERE name = 'Sita Rai';

SHOW INDEX FROM Student;
```

Compare the two `EXPLAIN` outputs — the `key` column changes from `NULL` to `idx_student_name`, and `rows` drops.

What you should see

`CALL` returning the right students; `@t` = 20000; `payment_log` containing a row you never inserted; and the two `EXPLAIN` outputs differing in the `key` column. Screenshot all four.

If it goes wrong

- "**You have an error in your SQL syntax near 'END'**": the delimiter — set the Delimiter box to `$$`, or paste the `DELIMITER $$ … DELIMITER ;` block exactly as written.
- "**PROCEDURE already exists**": `DROP PROCEDURE students_in_semester;` then create it again.
- **EXPLAIN looks the same:** with only 5 students the optimiser may skip the index — say so in your report; the `SHOW INDEX` output still proves the index exists.

What you learned

- **Stored procedure:** reusable logic stored *in* the database, called by name, can take `IN` / `OUT` parameters.
- **Trigger:** logic that fires **automatically** on INSERT/UPDATE/DELETE — used for audit logs.
- **Index:** makes searches on a column faster (like a book's index); costs extra storage and slightly slower writes.

PART 3 — Reliability and the project (Labs 13–16)

Lab 13 (Week 13) — Transactions & ACID

Goal: Group several statements so they **all succeed or all fail together**. Example: paying fees should reduce a balance *and* record a receipt — never one without the other.

Steps

1. Set up a simple balance table:

```
CREATE TABLE account (
  roll_no INT PRIMARY KEY,
  balance INT
);
INSERT INTO account VALUES (101, 20000), (102, 20000), (103, 20000);
```

2. A good transaction (COMMIT):

```
START TRANSACTION;
UPDATE account SET balance = balance - 15000 WHERE roll_no = 101;
INSERT INTO FeePayment VALUES (5006, 101, 15000, '2026-02-20');
COMMIT;   -- both steps are now permanent, together
```

Check: `SELECT * FROM account WHERE roll_no = 101;` → balance is 5000.

3. A rolled-back transaction (ROLLBACK): you notice a mistake before committing:

```
START TRANSACTION;
SELECT balance FROM account WHERE roll_no = 102;   -- 20000 before

-- oops, wrong amount
UPDATE account SET balance = balance - 99999 WHERE roll_no = 102;

-- see the temporary change
SELECT balance FROM account WHERE roll_no = 102;

ROLLBACK;   -- undo everything

SELECT balance FROM account WHERE roll_no = 102;   -- back to 20000
```

4. Atomicity in action — a transaction that fails halfway:

```
START TRANSACTION;
UPDATE account SET balance = balance - 5000 WHERE roll_no = 103;
-- ERROR: receipt 5001 exists
INSERT INTO FeePayment VALUES (5001, 103, 5000, '2026-02-21');
ROLLBACK;

-- still 20000: the deduction was undone too
SELECT balance FROM account WHERE roll_no = 103;
```

This is the whole point: the money was **not** taken, because the receipt could not be written.

5. SAVEPOINT — a partial undo:

```
START TRANSACTION;
UPDATE account SET balance = balance - 1000 WHERE roll_no = 101;
SAVEPOINT after_first;
UPDATE account SET balance = balance - 2000 WHERE roll_no = 102;
ROLLBACK TO after_first;   -- undo only the second update
COMMIT;                    -- keep the first
SELECT * FROM account;
```

What you should see

After COMMIT the change stays; after ROLLBACK it disappears; in step 4 the balance is untouched because the whole group failed. Screenshot the balance **before and after** the ROLLBACK, and the step-4 result.

If it goes wrong

- **ROLLBACK does nothing:** your table is MyISAM, which ignores transactions. Check with `SHOW TABLE STATUS WHERE Name = 'account';` — the Engine must be **InnoDB** (the default).
- **phpMyAdmin commits each statement separately:** paste the whole `START TRANSACTION … COMMIT;` block in **one** SQL box and run it once, not line by line.

What you learned

A transaction is **all-or-nothing** — the **A** in **ACID (Atomicity, Consistency, Isolation, Durability)**. `COMMIT` makes changes permanent; `ROLLBACK` undoes everything since `START TRANSACTION`; `SAVEPOINT` lets you undo just part. This is why banks and eSewa-style transfers use transactions.

Lab 14 (Week 14) — Concurrency: two users at once

Goal: See what happens when **two sessions** touch the same data, and how the database uses **locking** and **isolation levels** to keep things correct.

Steps

1. Open two SQL sessions. Easiest way: open phpMyAdmin in **two browser tabs** (or one normal + one incognito window). Call them **Tab A** and **Tab B**. Both should have `model_college` selected.

2. See a lock. - In **Tab A** run (and do *not* commit yet):

```
START TRANSACTION;
UPDATE account SET balance = balance - 1000 WHERE roll_no = 101;
```

- In **Tab B** run:

```
UPDATE account SET balance = balance - 500 WHERE roll_no = 101;
```

- **Tab B waits** (spins) — it is *blocked* because Tab A holds a lock on that row.
- Go back to **Tab A** and run `COMMIT;` — now **Tab B** completes. The database made them take turns so neither overwrites the other blindly.

3. Prove they didn't clash. Both deductions applied, in order:

```
SELECT balance FROM account WHERE roll_no = 101;
```

4. Rows lock, not whole tables. In Tab A start a transaction on roll 101 without committing, then in Tab B update **roll 102** — it goes through immediately, because it's a different row:

```
-- Tab A
START TRANSACTION;
UPDATE account SET balance = balance - 100 WHERE roll_no = 101;
-- Tab B (runs instantly, no waiting)
UPDATE account SET balance = balance - 100 WHERE roll_no = 102;
-- Tab A
COMMIT;
```

5. Isolation levels. Run in either session:

```
SELECT @@transaction_isolation;   -- usually REPEATABLE-READ
```

Note in your report: higher isolation (SERIALIZABLE) = safer but slower; lower (READ UNCOMMITTED) = faster but risks **dirty reads** (reading another transaction's uncommitted change).

6. See who is waiting (optional, run while Tab B is blocked):

```
-- the blocked session shows State = 'updating' / 'Waiting for lock'
SHOW PROCESSLIST;
```

What you should see

Tab B pausing until Tab A commits, then completing. Step 4 showing **no** wait for a different row. Screenshot both tabs — the waiting one, and then the completed one.

If it goes wrong

- **Tab B doesn't wait:** you forgot `START TRANSACTION` in Tab A, so its update auto-committed instantly and released the lock.
- **Tab B errors with "Lock wait timeout exceeded":** you waited too long (50 s default) — that is still a valid result; screenshot it and explain that the DBMS gave up waiting.
- **Both tabs share one session:** use an incognito window for Tab B.

What you learned

When many users share data the DBMS uses **locks** so changes don't clash — that is **concurrency control**. Locks are held until `COMMIT` and are taken **per row**, not per table. **Isolation levels** trade safety against speed. A **deadlock** is two transactions each waiting on the other; the DBMS detects it and cancels one.

Lab 15 (Week 15) — Backup, recovery & security

Goal: Make a **backup**, simulate a disaster, **restore** it, and set up basic **security** with users and privileges.

Steps

1. Back up (export). In phpMyAdmin, click the `model_college` database → **Export** tab → method **Quick** → format **SQL** → **Go**. A `.sql` file downloads. **This file is your backup** — it contains the SQL to rebuild everything. Open it in Notepad and look: it is just `CREATE TABLE` and `INSERT` statements.

2. Simulate a disaster. Run (carefully!):

```
DROP TABLE payment_log;   -- pretend this was lost
SELECT * FROM payment_log;   -- errors: the table is gone
```

3. Restore. Click the database → **Import** tab → **Choose file** → pick the `.sql` backup you just downloaded → **Go**. The dropped table returns. *(If it complains that other tables already exist, that's fine — the table you dropped is what matters; or restore into a fresh empty database to see a clean recovery.)*

```
SELECT * FROM payment_log;   -- back, with its rows
```

4. Security — least privilege. Create a limited user with SQL this time:

```
CREATE USER 'clerk'@'localhost' IDENTIFIED BY 'clerk123';

-- give only what is needed
GRANT SELECT, INSERT ON model_college.* TO 'clerk'@'localhost';
SHOW GRANTS FOR 'clerk'@'localhost';

-- take a right back
REVOKE INSERT ON model_college.* FROM 'clerk'@'localhost';
SHOW GRANTS FOR 'clerk'@'localhost';
```

5. Grant on a single table or view only — even tighter:

```
CREATE USER 'reader'@'localhost' IDENTIFIED BY 'read123';
GRANT SELECT ON model_college.paid_students TO 'reader'@'localhost';
SHOW GRANTS FOR 'reader'@'localhost';
```

`reader` can see the `paid_students` **view** but not the underlying `Student` table — this is the external level of the three-schema architecture, enforced.

6. Clean up (optional):

```
-- DROP USER 'reader'@'localhost';
```

What you should see

The `.sql` export file on your disk; the table gone after DROP and back after Import; `SHOW GRANTS` changing after the REVOKE. Screenshot the export dialog, the restored table, and both `SHOW GRANTS` outputs.

If it goes wrong

- **Import fails with "table already exists":** tick "**Enable foreign key checks: off**" in the Import options, or drop the database and import into an empty one.
- "**Access denied**" **creating a user:** you must be logged in as `root`.
- **The export downloads a `.sql.gz`:** set Compression to **None** in the Export options.

What you learned

Backups are your insurance — recovery means replaying the saved SQL. **Security** follows **least privilege**: give each user only the rights they need, with `GRANT` and `REVOKE`, and use a **view** to expose a slice of the data without the underlying table.

Lab 16 (Week 16) — Project demo

Goal: Present your complete Model College database as your **project** and explain any part of it on request.

Steps

1. Assemble your project (individual or a group of up to 4). It must show the full journey: - The **ER diagram** (Lab 3) - The **CREATE TABLE** script with constraints (Lab 4) and the **INSERT** data (Lab 5) - At least **6 queries**: a `WHERE`, an aggregate + `GROUP BY`, a `HAVING`, an `INNER JOIN`, a `LEFT JOIN`, and a subquery (Labs 6–9) - One **view**, one **stored procedure**, one **trigger**, one **index** (Labs 10, 12) - A **transaction** with `ROLLBACK` (Lab 13) and a **backup file** (Lab 15) - One **limited user** with `SHOW GRANTS` (Lab 15)

2. Run it live. Have phpMyAdmin open with `model_college` loaded. Be ready to run any query from your report on the spot — the examiner may pick one and ask you to change it (e.g. "now show only semester 4").

3. Present your project and answer questions on any part of it.

What you should see

A complete, working database you can explain end to end.

What you learned

You designed, built, populated, queried, joined, secured and backed up a real relational database. **This is the whole course in one project.**

Appendix A — Quick troubleshooting (all labs)

Problem	Cause	Fix
MySQL won't start (XAMPP)	Port 3306 in use / previous crash	Close other DB tools; restart PC; Start again
`http://localhost/phpmyadmin` not found	Apache not started	Start Apache in XAMPP
"Access denied for user root"	Typed a password	User `root`, password **empty** on fresh XAMPP
"Unknown column …"	Wrong database selected	Click `model_college` on the left first
"Foreign key constraint fails" (INSERT)	Inserted child before parent	Insert Teacher/Student/Course **before** Enrolment/FeePayment
"Cannot delete or update a parent row"	Child rows still reference it	Delete the children first
"Duplicate entry for PRIMARY"	Ran an INSERT twice	Don't re-run, or delete the duplicate row
`WHERE x = NULL` returns nothing	NULL isn't equal to anything	Use `IS NULL` / `IS NOT NULL`
"Column 'name' is ambiguous"	Two joined tables share a column name	Qualify it: `s.name`
JOIN returns a huge number of rows	Missing `ON` condition	Always give the `ON`
"Expression not in GROUP BY clause"	Selected a column that isn't grouped or aggregated	Add it to `GROUP BY` or wrap it in an aggregate
Procedure/trigger syntax error near `END`	Semicolons confuse phpMyAdmin	Set the **Delimiter** box to `$$`
`ROLLBACK` does nothing	Table engine is MyISAM	Use InnoDB (`SHOW TABLE STATUS`)
An `UPDATE` / `DELETE` changed everything	You forgot `WHERE`	`ROLLBACK` if in a transaction; else restore the Lab 15 backup

Appendix B — SQL you learned, in one place

DDL: `CREATE DATABASE`, `CREATE TABLE`, `PRIMARY KEY`, `FOREIGN KEY … REFERENCES`, `NOT NULL`, `CHECK`, `DEFAULT`, `SHOW TABLES`, `DESCRIBE`, `DROP TABLE` **DML:** `INSERT INTO … VALUES`, `UPDATE … SET … WHERE`, `DELETE FROM … WHERE` **Querying:** `SELECT … FROM … WHERE`, `AS`, `AND` / `OR` / `NOT`, `IN`, `BETWEEN`, `LIKE`, `ORDER BY`, `DISTINCT`, `LIMIT` **Summarising:** `COUNT` / `SUM` / `AVG` / `MIN` / `MAX`, `GROUP BY`, `HAVING`, `IS NULL` / `IS NOT NULL` **Combining:** `JOIN … ON`, `LEFT JOIN`, self-join, subqueries with `IN` / `NOT IN` / `EXISTS`, derived tables **Objects:** `CREATE VIEW`, `CREATE PROCEDURE` / `CALL`, `CREATE TRIGGER`, `CREATE INDEX`, `EXPLAIN` **Reliability:** `START TRANSACTION` / `COMMIT` / `ROLLBACK` / `SAVEPOINT`, `SHOW PROCESSLIST` **Security & backup:** `CREATE USER`, `GRANT`, `REVOKE`, `SHOW GRANTS`, Export/Import

> **Next:** answer the questions in *IT220_Lab_Questions.md*, lay them out using *IT220_Lab_Report_Format.md*, and get each lab signed off.